

LC 1122: Relative Sort Array

Authored by: Arpit Raj, LNMIIT Jaipur | Date: October 4, 2026

Problem Understanding

Conditions: arr 2 elements are distinct and all elements of arr 2 are also in arr 1.

Goal: now need to sort arr 1, st relative ordering of items in arr 1 is same as in arr 2. element that dont appear in arr 2 should be at end of arr 1, in ascending.

Example & Visualization

arr1: 2 3 1 3 2 4 6 7 9 2 19

arr2: 2 1 4 3 9 6

cnt Array Mapping (Frequency of arr1):

Index	0	1	2	3	4	5	6	7	8	9	...	19
Count	0	1	3	2	1	0	1	1	0	1	...	1

Constructing ans Array:

2, 2, 2, 1, 4, 3, 3, 9, 6, 7, 19

Approach: traverse arr 2, for each element look to cnt array, utilize them and reduce their frequency, after done traversing arr 2 traverse cnt array to take into consideration left overs

Method 1: Using Count Array

```
class Solution {
public:
    vector<int> relativeSortArray(vector<int>& arr1, vector<int>& arr2) {

        int n1 = arr1.size();
        int n2 = arr2.size();

        int maxu = *max_element(arr1.begin(), arr1.end());
        vector<int> cnt(maxu + 1, 0);

        for(int num1 : arr1){
            cnt[num1]++;
        }

        vector<int> ans;

        for(int num2 : arr2){
            while(cnt[num2]){
                ans.push_back(num2);
                cnt[num2]--;
            }
        }

        for(int i = 0; i <= maxu; i++){
            if(cnt[i] == 0) continue;
            while(cnt[i]){
                ans.push_back(i);
                cnt[i]--;
            }
        }

        return ans;
    }
};
```

Method 2: Using Standard Map

```
class Solution {
public:
    vector<int> relativeSortArray(vector<int>& arr1, vector<int>& arr2) {

        map<int, int> mp;

        for(int num1 : arr1){
            mp[num1]++;
        }

        vector<int> ans;

        for(int num2 : arr2){
            while(mp[num2]){
                ans.push_back(num2);
                mp[num2]--;
            }
        }

        for(auto it : mp){
            int num = it.first;

            while(mp[num]){
                ans.push_back(num);
                mp[num]--;
            }
        }

        return ans;
    }
};
```

This alternative approach uses an ordered `std::map` instead of a raw frequency array.